Here is the pseudocode for each of the 50 algorithm tasks, presented as a numbered list.

1. Determine whether two given strings are anagrams of each other.

```
text

FUNCTION areAnagrams(str1, str2):
    IF length(str1) != length(str2):
        RETURN False
    countArray = array of size 256 initialized to 0
    FOR each char in str1:
        countArray[ASCII(char)] = countArray[ASCII(char)] + 1
    FOR each char in str2:
        countArray[ASCII(char)] = countArray[ASCII(char)] - 1
        IF countArray[ASCII(char)] < 0:
            RETURN False
    RETURN True
```

2. Convert a non-negative decimal integer into its binary string representation.

```
text

FUNCTION decimalToBinary(n):
    IF n == 0:
        RETURN "0"
    binaryString = ""
    WHILE n > 0:
        remainder = n MOD 2
        binaryString = STRING(remainder) + binaryString
        n = n DIV 2
    RETURN binaryString
```

3. Determine whether a given year satisfies the leap year rules.

text

```
FUNCTION isLeapYear(year):
    IF (year MOD 400 == 0):
        RETURN True
    ELSE IF (year MOD 100 == 0):
        RETURN False
    ELSE IF (year MOD 4 == 0):
        RETURN True
    ELSE:
        RETURN False
```

4. Find the intersection of two sorted arrays and return the common elements.

text

```
FUNCTION intersectionSorted(arr1, arr2):
    i = 0, j = 0
    result = empty list
    WHILE i < length(arr1) AND j < length(arr2):
        IF arr1[i] == arr2[j]:
            result.append(arr1[i])
            i = i + 1
            j = j + 1
        ELSE IF arr1[i] < arr2[j]:
            i = i + 1
        ELSE:
            j = j + 1
    RETURN result
```

5. Remove punctuation from a sentence.

text

```
FUNCTION removePunctuation(sentence):
```

```
    punctuation = set(". , ! ? ; : ' \" - ( ) [ ] { }")
    result = ""
    FOR each char in sentence:
        IF char NOT IN punctuation:
            result = result + char
    RETURN result
```

6. Find the length of a given string.

```
text
FUNCTION stringLength(str):
    count = 0
    FOR each character in str:
        count = count + 1
    RETURN count
```

7. Split a sentence into individual words and store in a list.

```
text
FUNCTION splitIntoSentences(sentence):
    words = empty list
    currentWord = ""
    FOR each char in sentence:
        IF char == ' ':
            IF currentWord != "":
                words.append(currentWord)
                currentWord = ""
        ELSE:
            currentWord = currentWord + char
    IF currentWord != "":
        words.append(currentWord)
    RETURN words
```

8. Sort a list of words in alphabetical order.

text

```
FUNCTION sortWords(words):
   // Using a simple bubble sort for demonstration
   n = length(words)
   FOR i FROM 0 TO n-2:
      FOR j FROM 0 TO n-i-2:
         IF words[j] > words[j+1]:
            SWAP(words[j], words[j+1])
   RETURN words
```

9. Given an unsorted list of integers and a target value T, find the exact position of T in the list, or report that it doesn't exist.

text

```
FUNCTION linearSearch(list, T):
   FOR i FROM 0 TO length(list)-1:
      IF list[i] == T:
         RETURN i
   RETURN -1  // Indicates not found
```

10. Scan a paragraph of text and count how many times each individual vowel (a, e, i, o, u) appears.

text

```
FUNCTION countVowels(paragraph):
   vowelCount = {a:0, e:0, i:0, o:0, u:0}
   FOR each char in paragraph:
      lowerChar = toLower(char)
      IF lowerChar IN vowelCount:
         vowelCount[lowerChar] = vowelCount[lowerChar] + 1
   RETURN vowelCount
```

11. Given a list of IP addresses that accessed a server today, remove all duplicates and return only the unique IP addresses.

text

```
FUNCTION uniqueIPs(ipList):
   seen = empty set
   uniqueList = empty list
   FOR each ip in ipList:
      IF ip NOT IN seen:
         seen.add(ip)
         uniqueList.append(ip)
   RETURN uniqueList
```

12. Given a square grid of numbers (N x N), calculate the sum of the numbers that lie on the main diagonal from top-left to bottom-right.

text

```
FUNCTION diagonalSum(grid, N):
   sum = 0
   FOR i FROM 0 TO N-1:
      sum = sum + grid[i][i]
   RETURN sum
```

13. Reverse a string without using built-in reverse functions.

text

```
FUNCTION reverseString(str):
   reversed = ""
   FOR i FROM length(str)-1 DOWNTO 0:
      reversed = reversed + str[i]
   RETURN reversed
```

14. Find the sum of all even numbers in a list.

text

```
FUNCTION sumEven(list):
    sum = 0
    FOR each num in list:
        IF num MOD 2 == 0:
            sum = sum + num
    RETURN sum
```

15. Find the second largest in an array.

text

```
FUNCTION secondLargest(arr):
    IF length(arr) < 2:
        RETURN NULL // or error
    largest = -infinity
    second = -infinity
    FOR each num in arr:
        IF num > largest:
            second = largest
            largest = num
        ELSE IF num > second AND num != largest:
            second = num
    RETURN second
```

16. Find GCD of two numbers using Euclidean algorithm.

text

```
FUNCTION gcd(a, b):
    WHILE b != 0:
        temp = b
        b = a MOD b
        a = temp
```

```
    RETURN a
```

17. Compute the sum and average of a list of numbers.

```
text
FUNCTION sumAndAverage(list):
    sum = 0
    FOR each num in list:
        sum = sum + num
    average = sum / length(list)
    RETURN (sum, average)
```

18. Count the frequency of each character in a string.

```
text
FUNCTION charFrequency(str):
    freqMap = empty dictionary
    FOR each char in str:
        IF char IN freqMap:
            freqMap[char] = freqMap[char] + 1
        ELSE:
            freqMap[char] = 1
    RETURN freqMap
```

19. Swap two variables without using a temporary third variable.

```
text
FUNCTION swap(a, b):
    a = a + b
    b = a - b
    a = a - b
    RETURN (a, b)
```

20. Determine all divisors of a given positive integer.

text

```
FUNCTION getDivisors(n):
   divisors = empty list
   FOR i FROM 1 TO n:
      IF n MOD i == 0:
         divisors.append(i)
   RETURN divisors
```

21. Create a process that determines whether a number is even or odd.

text

```
FUNCTION isEven(n):
   IF n MOD 2 == 0:
      RETURN "Even"
   ELSE:
      RETURN "Odd"
```

22. Design a procedure that computes the factorial of a positive integer.

text

```
FUNCTION factorial(n):
   IF n == 0 OR n == 1:
      RETURN 1
   result = 1
   FOR i FROM 2 TO n:
      result = result * i
   RETURN result
```

23. Write a process that converts numerical scores into grade categories such as Pass, Merit, or Distinction.

text

```
FUNCTION convertGrade(score):
```

```
IF score >= 70:
    RETURN "Distinction"
ELSE IF score >= 60:
    RETURN "Merit"
ELSE IF score >= 50:
    RETURN "Pass"
ELSE:
    RETURN "Fail"
```

24. Create a process that checks whether a password contains at least 8 characters, one number, and one uppercase letter.

```
text
FUNCTION isPasswordStrong(password):
    IF length(password) < 8:
        RETURN False
    hasNumber = False
    hasUpper = False
    FOR each char in password:
        IF char IS digit:
            hasNumber = True
        IF char IS uppercase:
            hasUpper = True
    RETURN hasNumber AND hasUpper
```

25. Find the length of the longest substring without repeating characters in a given string.

```
text
FUNCTION longestSubstringLength(str):
    n = length(str)
    seen = empty set
    maxLen = 0
    left = 0
```

```
  FOR right FROM 0 TO n-1:
    WHILE str[right] IN seen:
      seen.remove(str[left])
      left = left + 1
    seen.add(str[right])
    maxLen = MAX(maxLen, right - left + 1)
  RETURN maxLen
```

26. Perform level-order traversal on a binary tree and return node values grouped by depth.

```
text
FUNCTION levelOrderTraversal(root):
  IF root IS NULL:
    RETURN empty list
  result = empty list
  queue = empty queue
  queue.enqueue(root)
  WHILE queue IS NOT empty:
    levelSize = queue.size()
    currentLevel = empty list
    FOR i FROM 0 TO levelSize-1:
      node = queue.dequeue()
      currentLevel.append(node.value)
      IF node.left != NULL:
        queue.enqueue(node.left)
      IF node.right != NULL:
        queue.enqueue(node.right)
    result.append(currentLevel)
  RETURN result
```

27. Rotate a square 2D matrix 90 degrees clockwise in-place.

text

```
FUNCTION rotateMatrix(matrix, N):
  // Transpose
  FOR i FROM 0 TO N-1:
    FOR j FROM i+1 TO N-1:
      SWAP(matrix[i][j], matrix[j][i])
  // Reverse each row
  FOR i FROM 0 TO N-1:
    FOR j FROM 0 TO N/2 -1:
      SWAP(matrix[i][j], matrix[i][N-1-j])
```

28. For a fixed window size sliding across an array, return the maximum value within each window position.

text

```
FUNCTION slidingWindowMax(arr, k):
  result = empty list
  deque = empty deque
  FOR i FROM 0 TO length(arr)-1:
    // Remove elements out of window from front
    WHILE deque IS NOT empty AND deque.front() <= i - k:
      deque.pop_front()
    // Maintain decreasing order in deque
    WHILE deque IS NOT empty AND arr[deque.back()] <= arr[i]:
      deque.pop_back()
    deque.push_back(i)
    // Add max to result once window is full
    IF i >= k - 1:
      result.append(arr[deque.front()])
  RETURN result
```

29. Build a text classification model to categorize text into predefined categories.

text

```
FUNCTION trainTextClassifier(documents, labels):
    // Step 1: Preprocess each document (tokenization, stopword removal, stemming)
    // Step 2: Create a vocabulary from the training set
    // Step 3: Convert each document to a feature vector (e.g., TF-IDF)
    // Step 4: Train a classifier (e.g., Naive Bayes, SVM, or Neural Network) on the feature vectors
    model = trainedModel
    RETURN model

FUNCTION predictCategory(model, newText):
    // Preprocess the new text similarly
    // Convert to feature vector
    // Use model to predict the category
    RETURN predictedCategory
```

30. Create a topic modelling system to extract underlying topics from a large corpus.

text

```
FUNCTION trainTopicModel(corpus):
    // Preprocess all documents (tokenization, remove stopwords, stemming)
    // Create a bag-of-words representation (document-term matrix)
    // Apply an algorithm like Latent Dirichlet Allocation (LDA)
    // Extract top words for each topic
    topicModel = trainedLDA
    RETURN topicModel

FUNCTION getTopics(topicModel, numberOfTopics):
    topics = empty list
    FOR each topic in topicModel:
        topWords = get top words for this topic
        topics.append(topWords)
```

```
    RETURN topics
```

31. Develop a co-reference resolution system to identify relationships between pronouns and nouns.

```
text

FUNCTION findCoreferences(text):
    // Parse the text to identify entities (nouns) and pronouns
    // Use syntactic and semantic rules (or a trained model)
    // Cluster mentions that refer to the same entity
    coreferenceChains = empty list of chains
    // Each chain is a list of mentions (pronouns and nouns) referring to the same entity
    RETURN coreferenceChains
```

32. Build a question answering system to answer user queries based on a given text.

```
text

FUNCTION answerQuestion(question, context):
    // Preprocess question and context
    // Use an information retrieval or a QA model (e.g., BERT-based)
    // Identify the most relevant span in the context that answers the question
    answer = extractedSpan
    RETURN answer
```

33. Implement a method to find the exact square root of a non-negative integer using binary search, rounded down to the nearest integer, without using built-in math functions.

```
text

FUNCTION integerSqrt(n):
    IF n < 2:
        RETURN n
    low = 1
    high = n
    ans = 0
    WHILE low <= high:
```

```
    mid = (low + high) DIV 2
    square = mid * mid
    IF square == n:
        RETURN mid
    ELSE IF square < n:
        ans = mid
        low = mid + 1
    ELSE:
        high = mid - 1
  RETURN ans
```

34. Find all elements that appear in two separate unsorted lists, ensuring the result contains no duplicates.

text

```
FUNCTION commonElements(list1, list2):
  set1 = set(list1)
  result = empty set
  FOR each item in list2:
    IF item IN set1:
      result.add(item)
  RETURN list(result)
```

35. Design a memory buffer that holds a fixed number of items. When it runs out of space, it must automatically discard the item that hasn't been accessed for the longest time. (LRU Cache)

text

```
CLASS LRUCache:
  // Use a combination of a dictionary and a doubly linked list (or an ordered map)
  cache = empty dictionary
  capacity = given capacity
  order = empty list (or a doubly linked list to maintain access order)

  FUNCTION get(key):
```

```
    IF key NOT IN cache:
        RETURN -1 // or NULL
    // Move key to front of order (most recently used)
    order.remove(key)
    order.insert_front(key)
    RETURN cache[key]

FUNCTION put(key, value):
    IF key IN cache:
        // Update value and move key to front
        cache[key] = value
        order.remove(key)
        order.insert_front(key)
    ELSE:
        IF length(order) >= capacity:
            // Evict the least recently used (last element in order)
            lruKey = order.back()
            order.remove_back()
            cache.remove(lruKey)
        order.insert_front(key)
        cache[key] = value
```

36. An ordered list of numbers was rotated at an unknown pivot point (e.g., [4,5,6,1,2,3]). Find the index of a target element in this list in logarithmic time.

text

```
FUNCTION searchRotatedArray(arr, target):
    low = 0
    high = length(arr) - 1
    WHILE low <= high:
        mid = (low + high) DIV 2
        IF arr[mid] == target:
```

```
        RETURN mid
    // Check if left half is sorted
    IF arr[low] <= arr[mid]:
        IF arr[low] <= target AND target <= arr[mid]:
            high = mid - 1
        ELSE:
            low = mid + 1
    // Right half must be sorted
    ELSE:
        IF arr[mid] <= target AND target <= arr[high]:
            low = mid + 1
        ELSE:
            high = mid - 1
  RETURN -1
```

37. Implement a stack using an array (push/pop/peek).

text

```
CLASS Stack:
  stackArray = empty array
  topIndex = -1

  FUNCTION push(item):
    topIndex = topIndex + 1
    stackArray[topIndex] = item

  FUNCTION pop():
    IF topIndex < 0:
      RETURN NULL // Stack underflow
    item = stackArray[topIndex]
    topIndex = topIndex - 1
```

```
        RETURN item

    FUNCTION peek():
        IF topIndex < 0:
            RETURN NULL
        RETURN stackArray[topIndex]
```

38. Find the first non-repeating character in a string.

```
text
FUNCTION firstNonRepeating(str):
    freqMap = empty dictionary
    FOR each char in str:
        freqMap[char] = freqMap.get(char, 0) + 1
    FOR each char in str:
        IF freqMap[char] == 1:
            RETURN char
    RETURN NULL // or special character
```

39. Detect cycle in a singly linked list (Floyd's cycle detection).

```
text
FUNCTION hasCycle(head):
    slow = head
    fast = head
    WHILE fast != NULL AND fast.next != NULL:
        slow = slow.next
        fast = fast.next.next
        IF slow == fast:
            RETURN True
    RETURN False
```

40. Implement a queue using two stacks.

```
text

CLASS QueueUsingStacks:
    stack1 = empty stack (for enqueue)
    stack2 = empty stack (for dequeue)

    FUNCTION enqueue(item):
        stack1.push(item)

    FUNCTION dequeue():
        IF stack2 IS empty:
            WHILE stack1 IS NOT empty:
                stack2.push(stack1.pop())
        IF stack2 IS empty:
            RETURN NULL // Queue is empty
        RETURN stack2.pop()
```

41. Solve the N Queens Problem for N arbitrary board size N.

```
text

FUNCTION solveNQueens(N):
    board = 2D array of size NxN initialized to 0
    solutions = empty list

    FUNCTION isSafe(board, row, col):
        // Check column
        FOR i FROM 0 TO row-1:
            IF board[i][col] == 1:
                RETURN False
        // Check upper-left diagonal
        i = row - 1, j = col - 1
```

```
    WHILE i >= 0 AND j >= 0:
      IF board[i][j] == 1:
        RETURN False
      i = i - 1, j = j - 1
    // Check upper-right diagonal
    i = row - 1, j = col + 1
    WHILE i >= 0 AND j < N:
      IF board[i][j] == 1:
        RETURN False
      i = i - 1, j = j + 1
    RETURN True

  FUNCTION placeQueen(board, row):
    IF row == N:
      solutions.append(copy of board)
      RETURN
    FOR col FROM 0 TO N-1:
      IF isSafe(board, row, col):
        board[row][col] = 1
        placeQueen(board, row + 1)
        board[row][col] = 0 // Backtrack

  placeQueen(board, 0)
  RETURN solutions
```

42. Find all pairs in an array that sum to a given target value.

```
text
FUNCTION findPairs(arr, target):
  seen = empty set
  result = empty list
```

```
  FOR each num in arr:
    complement = target - num
    IF complement IN seen:
      result.append((num, complement))
    seen.add(num)
  RETURN result
```

43. Solve the Tower of Hanoi problem for N discs.

```
text
FUNCTION towerOfHanoi(n, source, destination, auxiliary):
  IF n == 1:
    PRINT "Move disc 1 from " + source + " to " + destination
    RETURN
  towerOfHanoi(n-1, source, auxiliary, destination)
  PRINT "Move disc " + n + " from " + source + " to " + destination
  towerOfHanoi(n-1, auxiliary, destination, source)
```

44. Find the contiguous subarray with the maximum sum (Kadane's algorithm).

```
text
FUNCTION maxSubarraySum(arr):
  maxEndingHere = 0
  maxSoFar = -infinity
  FOR each num in arr:
    maxEndingHere = MAX(num, maxEndingHere + num)
    maxSoFar = MAX(maxSoFar, maxEndingHere)
  RETURN maxSoFar
```

45. Create instructions for adding two matrices of equal size.

```
text
FUNCTION addMatrices(A, B, rows, cols):
```

```
    result = matrix of size rows x cols
    FOR i FROM 0 TO rows-1:
        FOR j FROM 0 TO cols-1:
            result[i][j] = A[i][j] + B[i][j]
    RETURN result
```

46. Develop a process for multiplying two matrices.

```
text
FUNCTION multiplyMatrices(A, B, rowsA, colsA, colsB):
    // Assumes A is rowsA x colsA, B is colsA x colsB
    result = matrix of size rowsA x colsB initialized with 0
    FOR i FROM 0 TO rowsA-1:
        FOR j FROM 0 TO colsB-1:
            sum = 0
            FOR k FROM 0 TO colsA-1:
                sum = sum + A[i][k] * B[k][j]
            result[i][j] = sum
    RETURN result
```

47. Create an algorithm that updates inventory quantities after purchases and deliveries.

```
text
FUNCTION updateInventory(inventory, transactions):
    FOR each transaction in transactions:
        itemID = transaction.itemID
        quantityChange = transaction.quantity // positive for delivery, negative for purchase
        IF itemID IN inventory:
            inventory[itemID] = inventory[itemID] + quantityChange
        ELSE:
            // Handle new item, maybe set initial stock
            inventory[itemID] = quantityChange
```

```
    // Ensure no negative inventory (adjust accordingly)
    FOR each item in inventory:
        IF inventory[item] < 0:
            inventory[item] = 0 // or raise error
    RETURN inventory
```

48. Develop a recursive process that searches through nested folders to locate a specific file type.

```
text
FUNCTION findFilesByType(directory, fileExtension):
    results = empty list
    FOR each item in directory:
        IF item IS a file:
            IF item.extension == fileExtension:
                results.append(item.path)
        ELSE IF item IS a directory:
            // Recursive call
            subResults = findFilesByType(item, fileExtension)
            results.extend(subResults)
    RETURN results
```

49. Create a pathfinding solution that guides a user through a maze using graph traversal (BFS).

```
text
FUNCTION findPath(maze, start, end):
    // BFS traversal
    queue = empty queue
    visited = 2D boolean array same size as maze initialized to False
    parent = 2D array to store previous cell for reconstruction

    queue.enqueue(start)
    visited[start.row][start.col] = True
```

```
    directions = [(0,1), (1,0), (0,-1), (-1,0)] // right, down, left, up


    WHILE queue IS NOT empty:
        current = queue.dequeue()
        IF current == end:
            // Reconstruct path
            path = empty list
            WHILE current != start:
                path.append(current)
                current = parent[current.row][current.col]
            path.append(start)
            path.reverse()
            RETURN path
        FOR each direction in directions:
            newRow = current.row + direction.row
            newCol = current.col + direction.col
            IF newRow, newCol are within bounds AND maze[newRow][newCol] is not blocked AND
visited[newRow][newCol] == False:
                visited[newRow][newCol] = True
                parent[newRow][newCol] = current
                queue.enqueue((newRow, newCol))
    RETURN NULL // No path found
```

50. Develop algorithms enabling autonomous drones to coordinate movement while avoiding collisions.

```
text

// This is a simplified representation of a potential coordination algorithm


FUNCTION updateDronePositions(drones):
    // Each drone has a current position, velocity, and destination
    // 1. Communicate intentions: Each drone broadcasts its future path (e.g., next 3 positions)
```

```
  // 2. Detect potential collisions: For each pair of drones, check if their future paths intersect within a safe time horizon.

  // 3. Resolve conflicts: Assign priority (e.g., based on ID or distance to goal), lower priority drone adjusts its path/velocity to avoid collision.

  //    - Adjust velocity: Reduce speed, change heading slightly, or halt temporarily.

  // 4. Update positions: Move each drone according to its new path for this timestep.

  FOR each drone in drones:

    // Compute potential new position based on current vector

    proposedPosition = drone.position + drone.velocity

    // Check for conflicts (simplified)

    FOR each otherDrone in drones:

      IF distance(proposedPosition, otherDrone.position) < safeDistance:

        // Adjust velocity to avoid collision (e.g., move perpendicular or slow down)

        drone.velocity = calculateSafeVelocity(drone.position, otherDrone.position, drone.destination)

        proposedPosition = drone.position + drone.velocity

        break

    drone.position = proposedPosition

  RETURN drones
```